

FEDERAL UNIVERSITY OF TECHNOLOGY, OWERRI

DEPARTMENT OF SOFTWARE ENGINEERING

COURSE: SEN310

Policy Acknowledgment API

Technical Project Documentation

Prepared By:

Onyebueke Jesse Chibuikem

Registration Number: 20231370432

Serial Number: 193

Topic Number: 98

Project Overseen By:

Rev. Abraham Ovwonuri

Date: 24th August 2026

Table of Contents

1. Introduction and Project Overview
2. Technology Stack and Prerequisites
3. System Architecture
4. Project Structure (Folder and File Breakdown)
5. Development Steps
6. API Endpoint Specifications
7. The Database Model
8. Application Routing Architecture
9. Audit Tests
10. Error Handling and Validation
11. Method of Hosting
12. Current Stage of the Software
13. Limitations and Future Improvements
14. Conclusion
15. Appendix

Name: Onyebueke Jesse Chibuikem
Reg no: 20231370432
Course: SEN310
Dept: Software Engineering
Serial no: 193
Topic Number: 98
Date: 24th August 2026

1. Introduction and Project Overview

1.1 Project Title

Policy Acknowledgment API

1.2 Project Brief

The project brief required the development of a Policy Acknowledgment API using the Django web framework. The specification defined the following requirements:

- Type: API (Application Programming Interface)
- Required Endpoints: GET /current-policy and POST /acknowledge
- Validation: The system must store the policy version and a timestamp upon acknowledgment
- Submission: Audit tests must be written and submitted alongside the source code and screenshots.

1.3 What the Software Is Meant to Do

The Policy Acknowledgment API is a backend web service designed to manage the lifecycle of policy agreements between a system and its users. In real-world applications, organizations regularly update their Terms of Service, Privacy Policies, or internal compliance documents. It is legally and operationally important to maintain a verifiable record of who agreed to which version of a policy, and precisely when that agreement was made.

This API provides two core functions:

- It exposes the current version of a policy document to any requesting client application.
- It accepts and permanently records a user's acknowledgment of that policy, storing the user's identity, the version they agreed to, and an automatic timestamp of the moment agreement was received.

This type of system underpins many digital products; from employee onboarding platforms that require staff to acknowledge HR policies, to consumer applications that log user consent for GDPR compliance, to enterprise software that tracks internal policy signoffs for regulatory auditing purposes.

1.4 Repository and Source Code Access

The complete source code, automated test suite, database migration history, and technical documentation for this project are version-controlled and hosted on GitHub. This repository is structured to ensure reproducibility and readiness for collaborative environments. Evaluators and technical reviewers can access, clone, or inspect the live repository via the link below:

- **GitHub Repository:** [jesseonyebueke/policy-acknowledgment-api](https://github.com/jesseonyebueke/policy-acknowledgment-api): SEN310 API Project

2. Technology Stack and Prerequisites

The following technologies were used in the development of this project:

Technology	Purpose
Python	Core programming language
Django	Web framework providing project structure, ORM, and server
Django REST Framework (DRF)	Extension that enables building RESTful APIs with Django
SQLite	Default database used for storing acknowledgment records
pip	Python package manager used to install dependencies
Visual Studio Code	Integrated Development Environment (IDE) used for writing code

2.1 Why Django?

Django is a high-level Python web framework that follows the Model-View-Template (MVT) architectural pattern. I believe it was selected for this project because it provides a structured, opinionated approach to web development that enforces best practices, includes a built-in ORM (Object Relational Mapper) for database interactions, and ships with an automatic admin interface.

2.2 Why Django REST Framework?

Django REST Framework (DRF) is the industry-standard extension for building Web APIs on top of Django. It provides serialization, authentication, request parsing, and a browsable API interface that allows developers to interact with their API directly from a browser without needing a separate client application. DRF was used in this project to handle the `@api_view` decorator pattern, which simplifies the creation of function-based API views, and to return properly formatted JSON responses with correct HTTP status codes.

3. System Architecture

3.1 Architectural Pattern

This project follows the REST (Representational State Transfer) architectural style. REST is the dominant paradigm for designing web APIs. It defines a set of constraints for how systems should communicate over HTTP:

- Resources are identified by URLs (e.g., /api/current-policy/)
- HTTP methods define the action: GET retrieves data, POST submits data
- Responses are stateless; each request contains all information needed to process it
- Responses are formatted as JSON (JavaScript Object Notation), a lightweight data format

3.2 Request Flow Diagram

The following diagram illustrates how a request travels through the system from client to when response is returned:

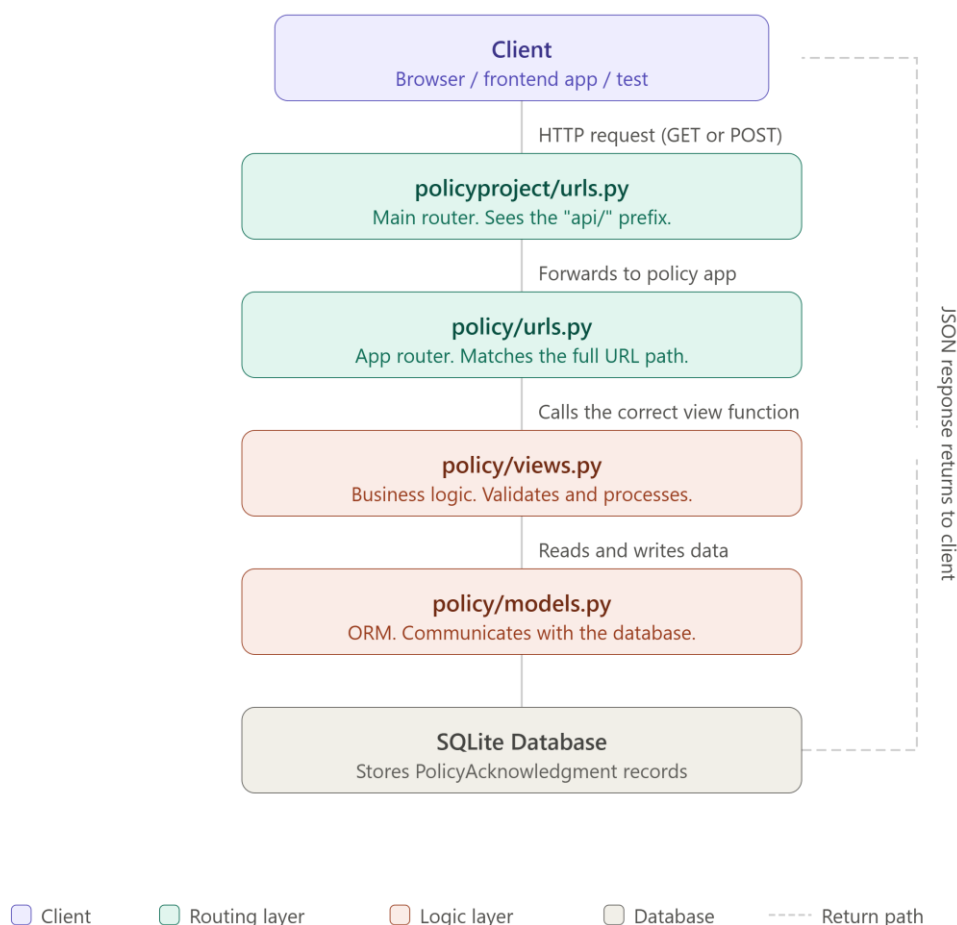


Fig1.1 - System Request Lifecycle and Architecture Diagram

4. Project Structure (Folder and File Breakdown)

4.1 Overview of the Directory Structure

When Django initializes a project, it creates a specific folder hierarchy. This project contains two top-level directories inside the working folder: `policyproject/` (the project configuration layer) and `policy/` (the application logic layer). Understanding the distinction between these two is fundamental to understanding Django.

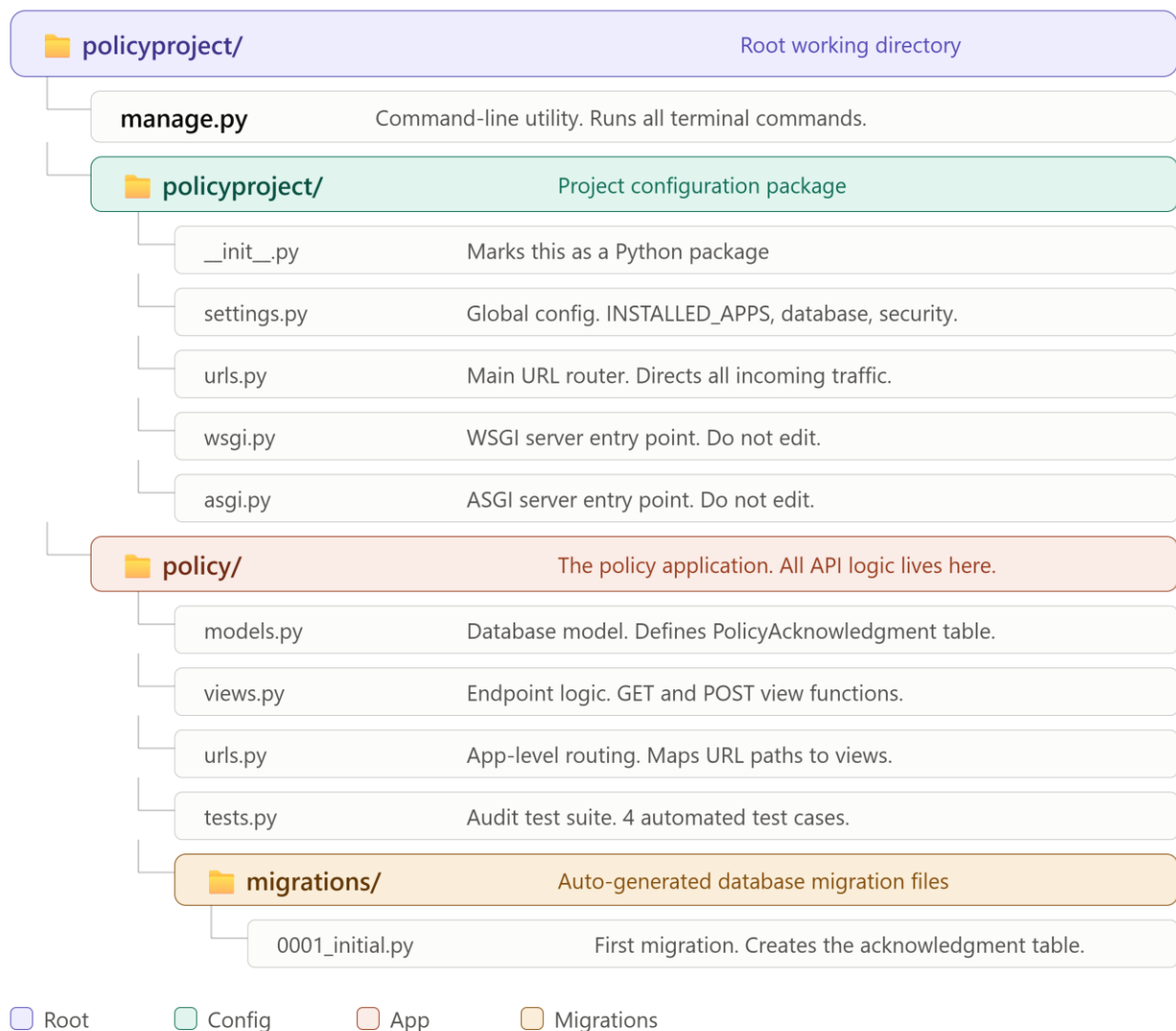


Fig1.2 - Project Structure Diagram

4.2 File-by-File Explanation

manage.py

This is Django's command-line utility file. It is the entry point for all terminal commands used during development. Every time a command such as `py manage.py runserver`, `py manage.py migrate`, or `py manage.py test policy` is executed, this file is what Python calls first. This file isn't edited; it exists purely to route administrative commands to the correct Django internals. It is automatically generated by Django and should never be deleted.

policyproject/settings.py

This is the global configuration file for the entire Django project. It functions as the control panel of the application. Key configurations defined here include: the list of installed applications (`INSTALLED_APPS`), the database configuration (defaulting to SQLite), security keys, middleware definitions, and static file settings. For this project, two critical entries were added to `INSTALLED_APPS`: `"rest_framework"` to enable Django REST Framework, and `"policy"` to register the policy application, so Django recognizes it as part of the project. Without these entries, Django would not load the application's models, views, or URLs.

policyproject/urls.py

This is the main URL routing file for the project. It acts as the primary reception desk where every HTTP request that reaches the Django server is first inspected here. This file contains a list called `urlpatterns`, which maps URL prefixes to their destinations. In this project, any URL beginning with `"api/"` is forwarded to the policy application's own `urls.py` file using Django's `include()` function. This design allows the project to scale: additional applications can each have their own URL files and be included here under different prefixes.

policyproject/wsgi.py and asgi.py

These files define the interface between the Django application and the web server that hosts it. WSGI (Web Server Gateway Interface) is the synchronous standard, while ASGI (Asynchronous Server Gateway Interface) supports asynchronous operations. For the purposes of this academic project, these files were not modified. They are invoked automatically when the development server starts. In a production deployment, these files would be the entry points for production-grade servers such as Gunicorn (WSGI) or Uvicorn (ASGI).

policy/models.py

This file contains the database model. In Django, a model is a Python class that inherits from `django.db.models.Model`. Each attribute of the class corresponds to a column in the database table. Django's ORM (Object Relational Mapper) reads this class definition and automatically creates and manages the corresponding database table. No SQL is written manually; Django translates the Python class into SQL statements internally.

policy/views.py

This is the most important file in the application. It contains the business logic that processes incoming HTTP requests and returns appropriate responses. Each function in this file is a view, decorated with `@api_view` to indicate which HTTP methods it accepts. The views retrieve and validate incoming data, interact with the database through the model, and return JSON responses with appropriate HTTP status codes.

policy/urls.py

This is the application-level URL routing file. While `policyproject/urls.py` handles top-level routing, this file handles routing within the policy application. It maps specific URL paths to specific view functions. This separation of routing concerns keeps the codebase organized and modular.

policy/tests.py

This file contains the automated audit test suite. Django's testing framework provides a `TestCase` class that allows developers to write automated tests that simulate HTTP requests and verify the responses without running a live server. These tests are executed via the terminal command `py manage.py test policy`, after which Django creates a temporary test database, runs all defined tests, reports the results, and destroys only the test database leaving the development database untouched.

policy/migrations/

This folder contains auto-generated migration files. When a developer defines or modifies a model and runs `py manage.py makemigrations`, Django generates a Python file in this folder that describes the database schema change. Running `py manage.py migrate` then applies those changes to the actual database. These files form a version history of the database schema and should be committed to version control alongside the source code.

5. Development Steps

5.1 Step 1: Project Initialization

The project was initialized using Django's command-line tool. The following commands were executed in sequence:

```
py -m django startproject policyproject
cd policyproject
py manage.py startapp policy
pip install djangoestframework
```

The first command creates the outer project directory and the inner configuration package. The second navigates into the project directory. The third creates the policy application directory with its default files. The fourth installs Django REST Framework from the Python Package Index (PyPI).

5.2 Step 2: Registering Applications

The `INSTALLED_APPS` list in `policyproject/settings.py` was updated to include both `"rest_framework"` and `"policy"`. This step is mandatory in Django, an application that is not registered in `INSTALLED_APPS` is invisible to the framework.

5.3 Step 3: Defining the Database Model

The `PolicyAcknowledgment` model was defined in `policy/models.py`. After saving the model file, two commands were run to create the database table:

```
py manage.py makemigrations
py manage.py migrate
```

The first command detects changes to model files and generates migration files. The second applies those migrations to the database, creating the physical table.

5.4 Step 4: Writing the Views

The two API view functions `"current_policy"` and `"acknowledge"` were written in `policy/views.py`.

The `CURRENT_POLICY` dictionary was defined at module level to serve as an in-memory representation of the current policy document.

5.5 Step 5: Configuring URL Routing

A new file `policy/urls.py` was created to map the two endpoint paths to their respective view functions. The main `policyproject/urls.py` was then updated to include the policy application's URLs under the `"api/"` prefix.

5.6 Step 6: Writing and Running Audit Tests

Four test cases were written in `policy/tests.py`. The tests were executed and verified to pass with the command:

```
py manage.py test policy
```

5.7 Step 7: Verification via Browser

The development server was started with `py manage.py runserver` and both endpoints were verified via the Django REST Framework's built-in browsable API interface at

<http://127.0.0.1:8000/api/current-policy/> for the GET Endpoint and
<http://127.0.0.1:8000/api/acknowledge/> for the POST Endpoint.

6. API Endpoint Specifications

6.1 Endpoint 1: GET /api/current-policy/

This endpoint allows any client to retrieve the current version of the policy document. It accepts only HTTP GET requests and requires no input data or authentication.

Property	Value
URL	http://127.0.0.1:8000/api/current-policy/
HTTP Method	GET
Authentication Required	No
Success Status Code	200 OK
Response Format	JSON

Successful response (200 OK):

```
HTTP 200 OK
Allow: GET, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "version": "1.0",
  "content": "By using this service, you agree to our terms and conditions.",
  "effective_date": "2026-06-01"
}
```

Fig 1.3 - GET Endpoint Successful JSON Response

6.2 Endpoint 2: POST /api/acknowledge/

This endpoint allows a client to submit a policy acknowledgment. It accepts only HTTP POST requests and requires a JSON body containing the user's name and the policy version they are acknowledging. The endpoint validates the input and, if valid, saves the acknowledgment to the database with an automatic timestamp.

Property	Value
URL	http://127.0.0.1:8000/api/acknowledge/
HTTP Method	POST
Authentication Required	No
Request Body	JSON with user_name and policy_version
Success Status Code	201 Created
Error Status Code	400 Bad Request
Response Format	JSON

Example request body:

```
{
  "user_name": "Onyebueke Jesse Chibuikem",
  "policy_version": "1.0"
}
```

Successful response (201 Created):

```
HTTP 201 Created
Allow: POST, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "message": "Policy acknowledged successfully",
  "user_name": "Onyebueke Jesse Chibuikem",
  "policy_version": "1.0",
  "acknowledged_at": "2026-08-22T13:15:32.658799Z"
}
```

Fig1.4 - Successful policy acknowledgement

Error response; missing fields (400 Bad Request):

```
HTTP 400 Bad Request
Allow: POST, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "error": "user_name and policy_version are required"
}
```

Fig1.5 - Error "Username and policy version are required"

Error response; wrong version (400 Bad Request):

```
HTTP 400 Bad Request
Allow: POST, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "error": "Invalid policy version"
}
```

Fig1.6 - Error "Invalid policy version"

7. The Database Model

7.1 Model Definition

The PolicyAcknowledgment model is defined in `policy/models.py` and represents a single row in the database table. Each instance of this model corresponds to one acknowledgment event, for example one user agreeing to one version of the policy at one specific moment in time.

```
from django.db import models

class PolicyAcknowledgment(models.Model):
    user_name = models.CharField(max_length=100)
    policy_version = models.CharField(max_length=20)
    acknowledged_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.user_name} acknowledged v{self.policy_version} at {self.acknowledged_at}"
```

7.2 Field Explanations

Field	Type	Description
user_name	CharField (max 100 chars)	The name of the user that acknowledges the policy. Stored as a string.
policy_version	CharField (max 20 chars)	The version string of the policy being acknowledged (e.g., "1.0").
acknowledged_at	DateTimeField (auto_now_add=True)	The exact date and time the acknowledgment was recorded. Set automatically by Django; the client cannot manipulate this value.

The `auto_now_add=True` parameter on the `acknowledged_at` field is a critical design decision. It means Django automatically sets this field to the current date and time when a new record is created, and it cannot be overridden by the client. This guarantees the integrity of the audit trail; users cannot backdate or falsify their acknowledgment timestamp.

8. Application Routing Architecture

8.1 Two-Level Routing System

Django uses a two-level URL routing system in this project. The main project `urls.py` handles top-level routing, and the application `urls.py` handles routing within the specific application.

8.2 `policyproject/urls.py` (Main Router)

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/", include("policy.urls")),
]
```

When a request arrives, Django checks this file first. If the URL begins with "admin/", it routes to Django's built-in admin interface. If it begins with "api/", it strips that prefix and forwards the remainder to `policy/urls.py`. The `include()` function is what enables this delegation.

8.3 `policy/urls.py` (Application Router)

```
from django.urls import path
from . import views

urlpatterns = [
    path("current-policy/", views.current_policy),
    path("acknowledge/", views.acknowledge),
]
```

After the "api/" prefix is stripped, this file receives the remainder of the URL. "current-policy/" maps to the `current_policy` view function, and "acknowledge/" maps to the `acknowledge` view function. The `from . import views` uses a relative import; The dot represents the current package (the policy application), ensuring `views.py` is loaded from the correct location.

9. Audit Tests

9.1 What Are Audit Tests?

Audit tests are automated verification scripts that confirm a system behaves exactly as specified, both under normal conditions and under erroneous input. Rather than manually testing each endpoint through a browser every time a change is made, automated tests can be run with a single command and will consistently verify every specified behavior in milliseconds. This is a foundational practice in professional software development known as Test-Driven Development (TDD) and is a mandatory deliverable for this project.

9.2 Test Infrastructure

Django provides a built-in testing framework based on Python's unittest library. Django REST Framework extends this with APIClient, A specialised test client that can send HTTP requests to API endpoints and inspect the responses without running a live server. Django also creates a separate, temporary test database for each test run, ensuring that test data does not contaminate the development database.

9.3 The Four Test Cases

Test 1: test_get_current_policy

This test verifies that the GET `/api/current-policy/` endpoint responds correctly under normal conditions. It works by sending a GET request to `/api/current-policy/` and asserts that the response status code is 200 (OK), and that the response body contains the fields "version", "content", and "effective_date". This confirms that the endpoint is reachable and returns a properly structured policy document. If this test fails, the GET endpoint is broken.

Test 2: test_acknowledge_success

This verifies that the POST `/api/acknowledge/` endpoint correctly saves a valid acknowledgment to the database. It works by sending a POST request with a valid `user_name` ("Onyebueke Jesse Chibuikem") and a valid `policy_version` ("1.0"). Asserts that the response status code is 201 (Created) and that the response body contains the "acknowledged_at" field. This confirms that a valid acknowledgment has persisted to the database, and the timestamp is returned. If this test fails, the audit trail is broken.

Test 3: test_acknowledge_missing_fields

This test verifies that the POST endpoint correctly rejects incomplete requests. It works by sending a POST request with an empty body (no `user_name`, no `policy_version`). Asserts that the response status code is 400 (Bad Request). Incomplete data must never be silently accepted and stored. This test ensures the validation logic correctly identifies and rejects malformed requests.

Test 4: test_acknowledge_wrong_version

It verifies that the POST endpoint correctly rejects acknowledgments for non-existent policy versions. It sends a POST request with a valid `user_name` but an invalid `policy_version` ("9.9"). Asserts that the response status code is 400 (Bad Request). Users should only be able to acknowledge the current policy version. Accepting an outdated or fabricated version would undermine the integrity of the audit trail.

9.4 Test Results

All four tests passed successfully on execution:

```
(.venv) PS C:\Users\DELL\Desktop\policyproject> py manage.py check
>> py manage.py test policy
System check identified no issues (0 silenced).
Found 4 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
....
-----
Ran 4 tests in 0.011s

OK
Destroying test database for alias 'default'...
(.venv) PS C:\Users\DELL\Desktop\policyproject>
```

Fig1.7 - Automated test suite execution output on the VS code terminal

The four dots in the output each represent one passing test. A failing test would display an "F" and an error would display an "E". The "OK" at the end confirms all tests passed without exception.

Test Name	HTTP Method	Input	Expected Status	Result
test_get_current_policy	GET	None	200 OK	PASSED
test_acknowledge_success	POST	Valid user + version	201 Created	PASSED
test_acknowledge_missing_fields	POST	Empty body	400 Bad Request	PASSED
test_acknowledge_wrong_version	POST	Wrong version "9.9"	400 Bad Request	PASSED

10. Error Handling and Validation

10.1 Validation Logic in the POST Endpoint

The acknowledge view implements two sequential validation checks before any database operation is performed:

Check 1 (Presence Validation): The view calls `request.data.get()` for both `user_name` and `policy_version`. If either value is absent or empty (evaluates falsy in Python), the view immediately returns a 400 Bad Request response with a descriptive error message. No database write is attempted.

Check 2 (Version Validation): The submitted `policy_version` is compared against the version defined in the `CURRENT_POLICY` dictionary. If they do not match, the view returns a 400 Bad Request response indicating an invalid version. This prevents users from acknowledging outdated or non-existent policy versions.

10.2 HTTP Status Codes Used

Status Code	Name	When It Is Returned
200	OK	GET /api/current-policy/ succeeds
201	Created	POST /api/acknowledge/ successfully saves a record
400	Bad Request	POST request has missing fields or wrong policy version
404	Not Found	A URL that does not exist is requested (Django default)
405	Method Not Allowed	A POST is sent to the GET endpoint or vice versa (DRF default)

10.3 Common Errors Encountered During Development

Error	Cause	Resolution
404 on /api/current-policy/	"policy" not in <code>INSTALLED_APPS</code> or <code>include()</code> missing in main <code>urls.py</code>	Verified both <code>urls.py</code> files and <code>settings.py</code> registration
<code>ModuleNotFoundError: rest framework</code>	<code>djangoRESTframework</code> not installed	Ran <code>pip install djangoRESTframework</code>
"from django.urls" syntax error	Missing "f" in "from"; typo in <code>policy/urls.py</code>	Corrected the import statement
<code>CommandError: policy_project conflicts</code>	Project name matched an existing Python module	Renamed project to <code>policyproject</code> (no underscore)
"py" not recognised	Python not in system <code>PATH</code> or wrong alias	Used "py" instead of "python" on Windows
Server shows only admin/ in URL patterns	<code>include()</code> not imported or <code>policy.urls</code> not included	Fixed the import and <code>urlpatterns</code> in main <code>urls.py</code>

11. Method of Hosting

11.1 Development Server

This project is currently hosted using Django's built-in development server, launched with the command:

```
py manage.py runserver
```

This starts a lightweight HTTP server that listens on localhost (127.0.0.1) at port 8000. The address 127.0.0.1 is a loopback address meaning that it refers exclusively to the local machine. This means the API is only accessible from the same computer on which the server is running; it is not exposed to the internet or any external network.

The development server is appropriate for testing and demonstration purposes. It features automatic code reloading (changes to Python files are detected, and the server restarts automatically), detailed error pages when `DEBUG=True`, and a browsable API interface provided by Django REST Framework.

11.2 Technical Hosting Terminology

The full technical name for what is being run is: a Django REST API application served by a WSGI development server, running on a localhost loopback address on port 8000.

11.3 What Would Production Hosting Look Like?

In a production environment, this application would require the following changes:

- `DEBUG` would be set to `False` in `settings.py`
- `ALLOWED_HOSTS` would be configured with the production domain name
- A production-grade WSGI server such as Gunicorn would replace the development server
- A reverse proxy such as Nginx would sit in front of Gunicorn to handle static files and SSL termination
- The database would be migrated from SQLite to a production database such as PostgreSQL
- The application would be deployed to a cloud platform such as AWS, Heroku, or DigitalOcean
- Environment variables would be used to manage secret keys and database credentials securely

12. Current Stage of the Software

12.1 Current capabilities

The following capabilities have been fully implemented and verified:

Capability	Status
Expose current policy via GET /api/current-policy/	Fully implemented and tested
Accept policy acknowledgment via POST /api/acknowledge/	Fully implemented and tested
Validate presence of required fields (user_name, policy_version)	Fully implemented and tested
Validate that submitted version matches current policy version	Fully implemented and tested
Persist acknowledgment record to database with auto-timestamp	Fully implemented and tested
Return appropriate HTTP status codes for all scenarios	Fully implemented and tested
Browsable API interface for manual testing	Available via DRF
4 automated audit tests, all passing	Fully implemented and passing

12.2 Gap Analysis

When compared against the requirements of a production-ready policy acknowledgment system, the following limitations exist in the current implementation.

Missing Feature	Impact	Effort to Add
User authentication	Any request with any name is accepted; identity cannot be verified	Medium: DRF provides TokenAuthentication
Multiple policy versions	Only one hardcoded policy exists; no way to add new policies without editing code	Medium: requires a Policy model
Retrieve acknowledgment history	No endpoint to query who has acknowledged which versions	Low: add a GET /acknowledgments/ endpoint
Duplicate acknowledgment prevention	A user can acknowledge the same policy multiple times	Low: add a unique_together constraint to the model
Pagination	If thousands of records exist, all are returned at once	Low: DRF provides built-in pagination
Production database (PostgreSQL)	SQLite is not suitable for concurrent production use	Medium: requires server configuration
Serializer classes	Data validation uses manual .get() calls instead of DRF Serializers	Low: improves code quality and extensibility
API documentation (Swagger/OpenAPI)	No auto-generated documentation for external consumers	Low: drf-spectacular package

13. Limitations and Future Improvements

The current implementation satisfies the project's brief in full. However, some minor enhancements would be required before this system could be deployed in a real-world scenario:

13.1 Authentication and Authorization

The current API does not verify the identity of the requesting user. Any client can submit any `user_name` string, and it will be accepted. A production system would integrate Django's authentication system or DRF's token-based authentication to ensure that only registered, logged-in users can submit acknowledgments, and that the `user_name` is derived from the authenticated session rather than supplied by the client.

13.2 Policy Management

The current policy content and version are hardcoded as a Python dictionary in `views.py`.

A production system would store policies in the database as a separate model, allowing authorized administrators to create, update, and version policies through an admin interface or dedicated API endpoints.

13.3 Serializer Classes

Django REST Framework provides a Serializer class system that handles data validation, transformation, and database interaction in a structured, reusable way. The current implementation performs validation manually using `if` statements. Introducing DRF Serializers would improve code quality, reduce the risk of validation errors, and align the codebase with industry best practices.

13.4 Comprehensive Error Handling

The current implementation handles the two most common error cases. A production system would implement global exception handlers to catch unexpected errors, return consistent error response formats, and log errors to a monitoring service such as Sentry.

14. Conclusion

This project successfully delivered a functional Policy Acknowledgment REST API using Django and Django REST Framework, in fulfilment of the academic project brief. The system implements the two required endpoints; GET /api/current-policy/ and POST /api/acknowledge/ with appropriate data validation, database persistence, and HTTP status code responses.

The four audit tests written in policy/tests.py comprehensively verify the correct behavior of both endpoints under both valid and invalid input conditions, and all four pass without error. The automated test suite constitutes the primary mechanism of quality assurance and serves as executable documentation of the system's intended behavior.

From an architectural standpoint, the two-level URL routing system, the ORM-based database model with automatic timestamping, and the DRF-based view functions collectively form a coherent, extensible backend service.

This current implementation is appropriately scoped for testing and demonstration purposes. The gap analysis in Section 12.2 identifies the specific enhancements (Authentication, policy versioning, serializers, and production database configuration) that would be required to elevate this system to production readiness. These represent natural extensions of the existing architecture rather than fundamental redesigns, demonstrating that the foundational decisions made in this implementation are sound.

In summary, this Policy Acknowledgment API meets all specified requirements: it exposes the current policy via a GET endpoint, records acknowledgments with version and timestamp via a POST endpoint, validates inputs, and is accompanied by a passing suite of four audit tests.

15. Appendix

15.1 policy/models.py

```
from django.db import models

class PolicyAcknowledgment(models.Model):
    user_name = models.CharField(max_length=100)
    policy_version = models.CharField(max_length=20)
    acknowledged_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.user_name} acknowledged v{self.policy_version} at {self.acknowledged_at}"
```

15.2 policy/views.py

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status
from .models import PolicyAcknowledgment

CURRENT_POLICY = {
    "version": "1.0",
    "content": "By using this service, you agree to our terms and conditions.",
    "effective_date": "2026-06-01"
}

@api_view(["GET"])
def current_policy(request):
    return Response(CURRENT_POLICY)

@api_view(["POST"])
def acknowledge(request):
    user_name = request.data.get("user_name")
    policy_version = request.data.get("policy_version")

    if not user_name or not policy_version:
        return Response(
            {"error": "user_name and policy_version are required"},
            status=status.HTTP_400_BAD_REQUEST
        )

    if policy_version != CURRENT_POLICY["version"]:
```

```

        return Response(
            {"error": "Invalid policy version"},
            status=status.HTTP_400_BAD_REQUEST
        )

    acknowledgment = PolicyAcknowledgment.objects.create(
        user_name=user_name,
        policy_version=policy_version
    )

    return Response({
        "message": "Policy acknowledged successfully",
        "user_name": acknowledgment.user_name,
        "policy_version": acknowledgment.policy_version,
        "acknowledged_at": acknowledgment.acknowledged_at
    }, status=status.HTTP_201_CREATED)

```

15.3 policy/urls.py

```

from django.urls import path
from . import views

urlpatterns = [
    path("current-policy/", views.current_policy),
    path("acknowledge/", views.acknowledge),
]

```

15.4 policyproject/urls.py

```

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/", include("policy.urls")),
]

```

15.5 policy/tests.py

```

from django.test import TestCase
from rest_framework.test import APIClient

class PolicyAPITests(TestCase):

    def setUp(self):

```

```

        self.client = APIClient()

    def test_get_current_policy(self):
        response = self.client.get("/api/current-policy/")
        self.assertEqual(response.status_code, 200)
        self.assertIn("version", response.data)
        self.assertIn("content", response.data)
        self.assertIn("effective_date", response.data)

    def test_acknowledge_success(self):
        response = self.client.post("/api/acknowledge/", {
            "user_name": "hiro",
            "policy_version": "1.0"
        }, format="json")
        self.assertEqual(response.status_code, 201)
        self.assertIn("acknowledged_at", response.data)

    def test_acknowledge_missing_fields(self):
        response = self.client.post("/api/acknowledge/", {}, format="json")
        self.assertEqual(response.status_code, 400)

    def test_acknowledge_wrong_version(self):
        response = self.client.post("/api/acknowledge/", {
            "user_name": "hiro",
            "policy_version": "9.9"
        }, format="json")
        self.assertEqual(response.status_code, 400)

```

15.6 policyproject/settings.py — INSTALLED_APPS (relevant section)

```

INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "rest_framework",    # Django REST Framework
    "policy",            # Policy acknowledgment app
]

```